# LAB 5: Data Integration

**Student:** Dorline TIAKO OKASSIE | ING4 Data & Artificial Intelligence – Group 01 | ECE Paris

*Tool: Talend Open Studio for Data Integration 8.0.1 | Input files: movies.csv, Directors.txt*

## Exercise 1:

This exercise describes a simple data integration job that reads movie-related data from a locally stored CSV file and displays the data in the Run view console (Extract ETL step).

**Project Creation:**

1. Launch Talend.

2. Create a new project named PW1.

3. Select the PW1 project.

### Answer – Project Creation

Talend Open Studio was launched and, in the project selection window, the option **Create a project** was used to create a new local project named **PW1**. The project was then selected and opened. The title bar of the studio confirms that the active project is **PW1 (Connexion: Local)**, and the Repository shows the standard folders (Jobs, Contexts, Code, SQL Templates, Metadata, Documentation, Recycle bin).

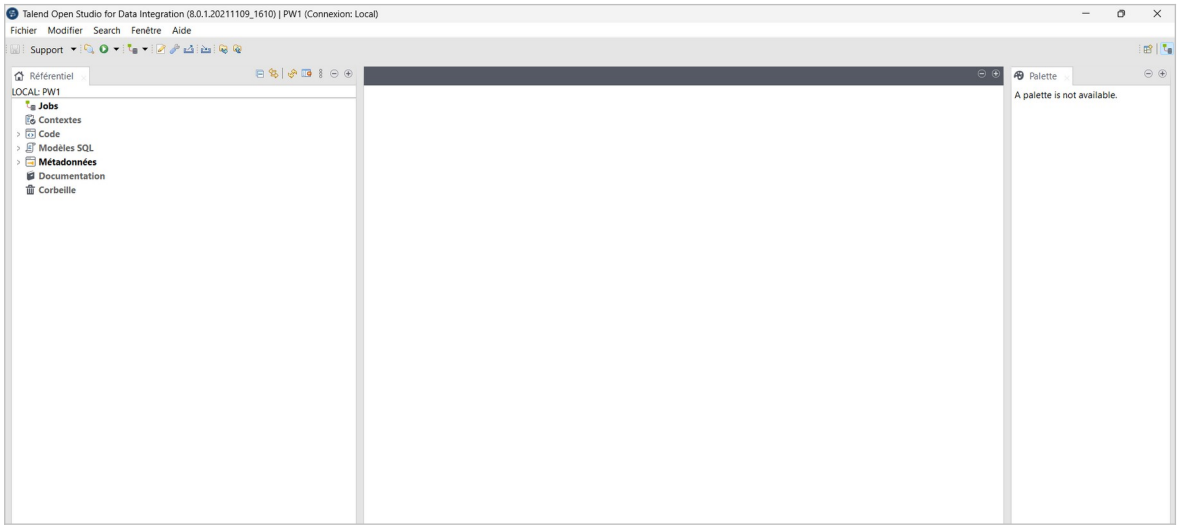



***Figure 1:*** *Project PW1 opened in Talend Open Studio (title bar: "PW1 (Connexion: Local)")*

© 2026 Caplogy. All rights reserved.

**Metadata Creation:**

1. Create the metadata for the movies.csv file (Delimited File).

2. Create a delimited file named movies.

3. Browse the file.

4. Set the field separator to ";".

5. Enable the Define Header Line option.

6. Click Next and Finish.

## Answer – Metadata Creation

A **Delimited File** metadata named **movies** was created in the Repository (Metadata › File delimited › Create file delimited). The wizard has four steps:

- **Step 1 – Properties:** name `movies`, purpose “Read movies data from movies.csv”.
- **Step 2 – File:** the file `movies.csv` was selected with the Browse button. Talend detected the **UNIX** format (the file uses LF line endings), which was kept.
- **Step 3 – Parsing:** field separator set to **Semicolon (";")**, row separator “Standard EOL”, and the option **Set heading row as column names** enabled (header = 1 line ignored). The preview shows the five real column names.
- **Step 4 – Schema:** five columns were detected. `directorID` was kept as **String** so that it can later be joined with the `director_id` column of the directors file, and all columns are nullable (many movies have an empty directorID).

© 2026 Caplogy. All rights reserved.

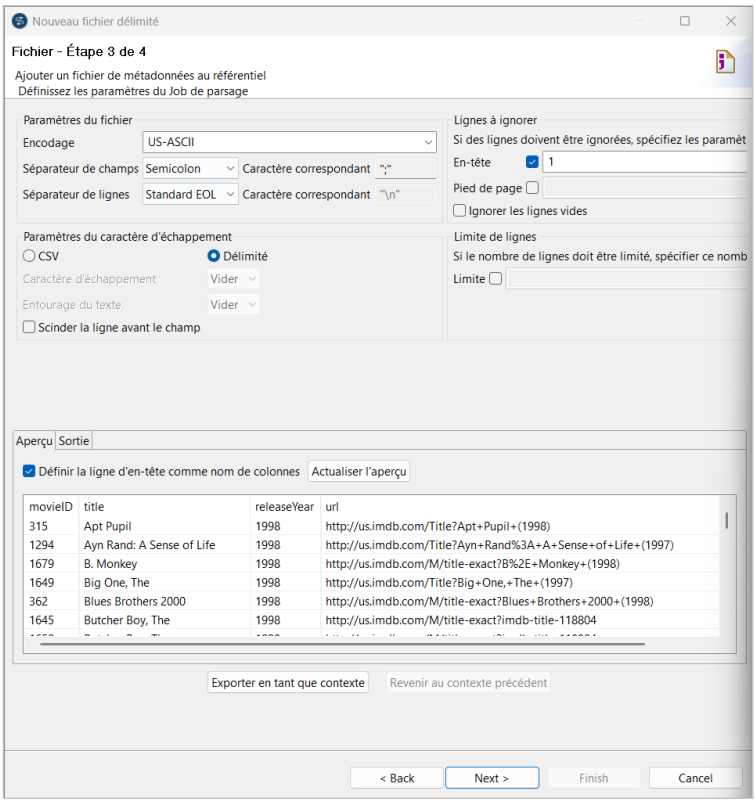



***Figure 2:*** *Metadata "movies" – step 3: semicolon separator and header line enabled*

| Column | Type | Description |
|---|---|---|
| movieID | Integer | Unique identifier of the movie |
| title | String | Title of the movie |
| releaseYear | Integer | Year of release |
| url | String | IMDb link |
| directorID | String | Foreign key to the director (empty for 1,200 movies) |

© 2026 Caplogy. All rights reserved.

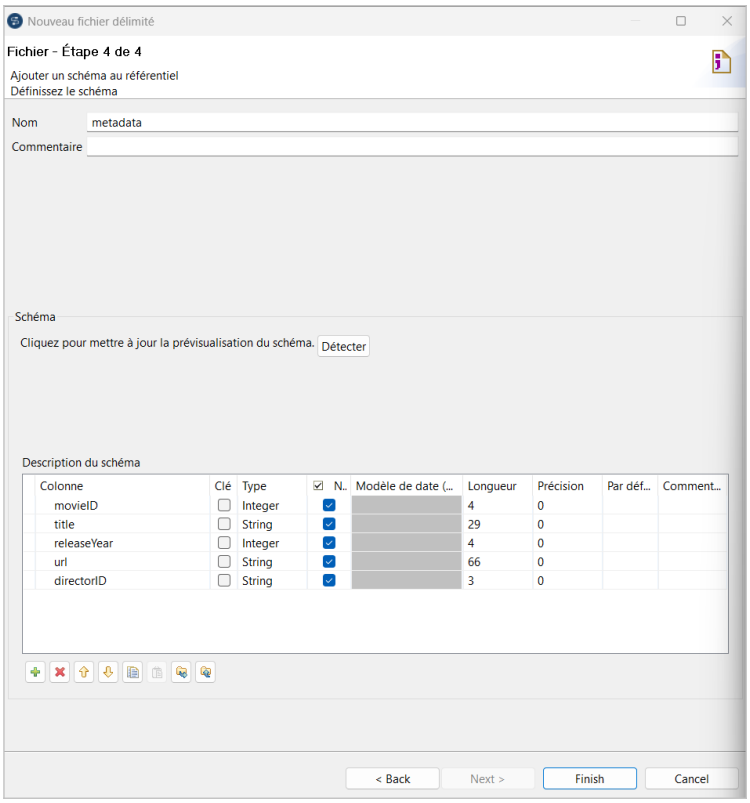



***Figure 3:*** *Metadata "movies" – step 4: schema of the file*

**Job Creation**

1. Create a Job Exercise 1:

2. Drag the movies file and choose TFileInputDelimited.

3. Drag a TLogROW component.

4. Connect movies to TLogROW with a Main link.

5. Execute the job

6. Document the job.

## Answer – Job Creation, Execution and Documentation

A job named **Exercise1** was created. The **movies** metadata was dragged onto the design workspace as a **tFileInputDelimited** component, a **tLogRow** component was added from the Palette, and both were connected with a **Row › Main** link (row1). The tLogRow was set to **Table** mode to obtain a readable output in the console.

© 2026 Caplogy. All rights reserved.

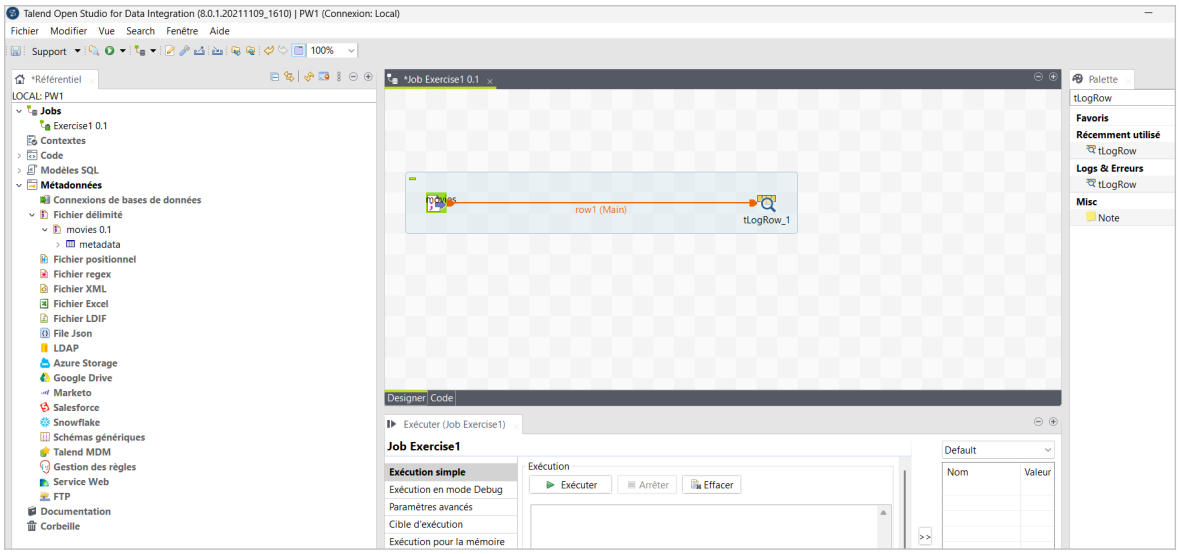



*Figure 4: Job Exercise1 – movies (tFileInputDelimited) connected to tLogRow with a Main link*

The job was executed from the **Run** view. The statistics on the link show **1455 rows** read in about 0.1 s, and the console displays every movie of the file. The job ends with **exit code 0** (no error).

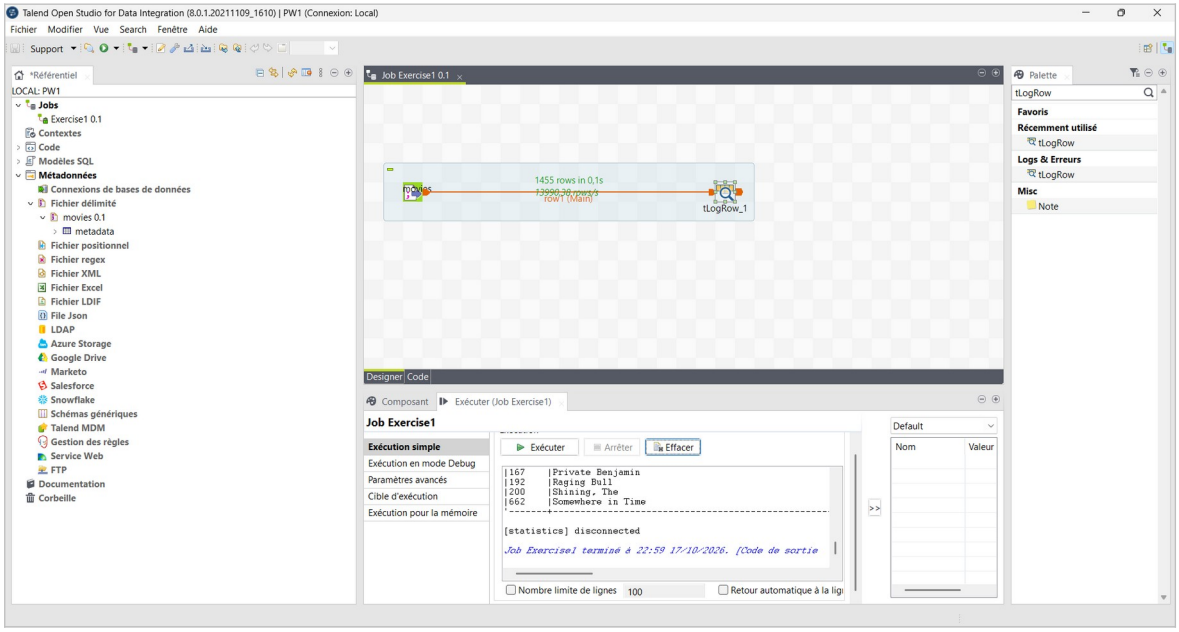



*Figure 5: Execution of Exercise1: 1455 rows displayed in the Run console (exit code 0)*

## Job documentation

| Component | Role | Main settings |
|---|---|---|
| movies (tFileInputDelimited) | Extract – reads the source file | Repository metadata `movies`, separator ";", header = 1, schema of 5 columns |
| row1 (Main) | Data flow | Carries the 5 columns of each movie |
| tLogRow_1 | Displays the data | Mode Table, output in the Run console |

© 2026 Caplogy. All rights reserved.

> **Result:** this job illustrates the **Extract** step of the ETL process: the 1,455 movies of the CSV source are accessed through a reusable metadata definition and displayed without any transformation.

## Exercise 2:

This exercise describes how to filter the data stream (Movies) and obtain only valid movies with valid information about directors (Transform ETL step).

1. Create a Job Exercise 2

2. Drag the movies file and choose TFileInputDelimited.

3. Create a delimited file named directors.

4. Browse the file.

5. Set the field separator to ",".

6. Change column names to director_id and director_name.

7. Change the type of director_id to String.

8. Drag the directors file and choose TFileInputDelimited.

9. Drag a TMap component.

10. Connect movies to TMap with a Main link.

11. Connect directors to TMap with a Main link.

12. Drag a TLogROW component.

13. Connect TMap to TLogROW with an output Main link named output.

14. Double-click on the TMap.

15. Join the director_id column from movies with the director_id column from directors.

16. Set the JoinModel option to "Inner Join".

17. Transfer the title columns from movies and director_name columns from directors to the output stream output.

18. Click Apply and OK.

19. Execute the job

© 2026 Caplogy. All rights reserved.

20. Document the job

## Answer – Exercise 2

### 1. Metadata of the directors file (steps 3 to 7)

A second **Delimited File** metadata named **directors** was created for `Directors.txt`. Unlike movies.csv, this file uses **WINDOWS** line endings (CRLF), has **no header line**, and its separator is a **comma**. The encoding was set to **UTF-8**.

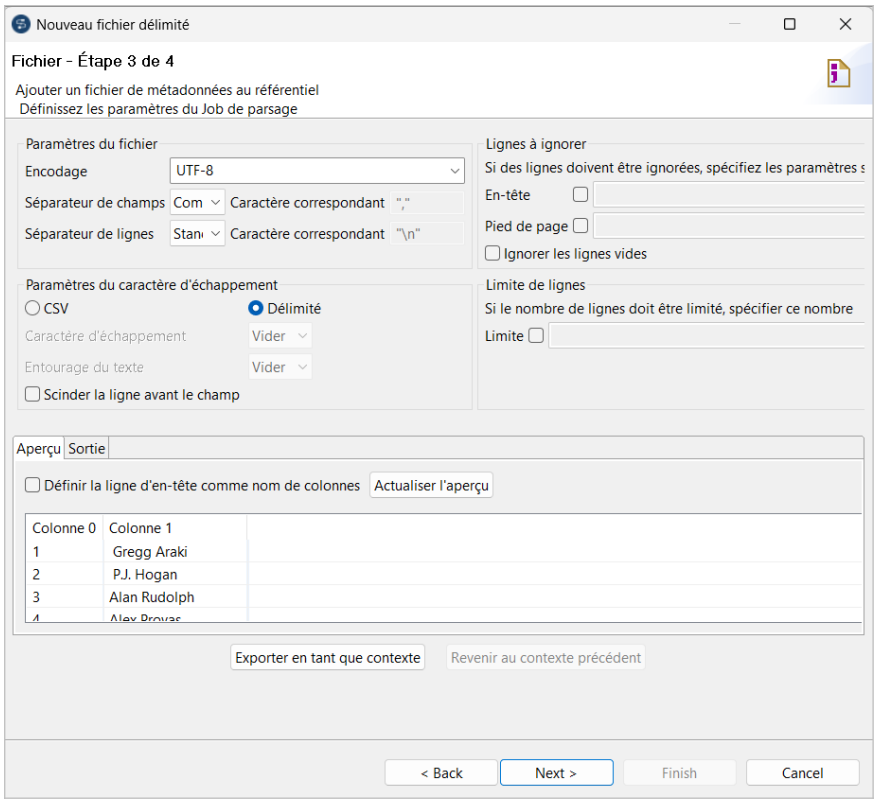



***Figure 6:*** *Metadata "directors" – step 3: comma separator, no header, two columns detected*

In the schema step, the default columns "Column0" and "Column1" were renamed **director_id** and **director_name**, and the type of `director_id` was set to **String** (same type as `directorID` in movies, which is required for the join).

© 2026 Caplogy. All rights reserved.

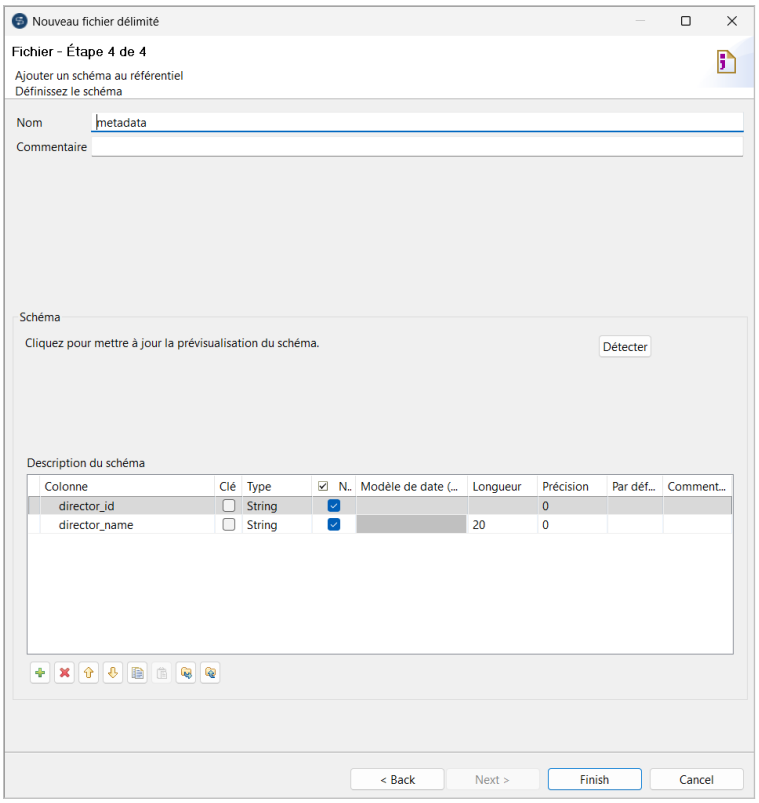



***Figure 7:*** *Metadata "directors" – step 4: columns renamed and director_id set to String*

## 2. Design of the job (steps 8 to 13)

In the job **Exercise2**, the movies and directors metadata were added as two **tFileInputDelimited** components and connected to a single **tMap**. The first connection (movies) becomes the **main flow – row1 (Main)**, and the second one (directors) automatically becomes the **lookup flow – row2 (Lookup)**, drawn as a dotted line. The tMap is connected to a **tLogRow** through a new output named **output**.

© 2026 Caplogy. All rights reserved.

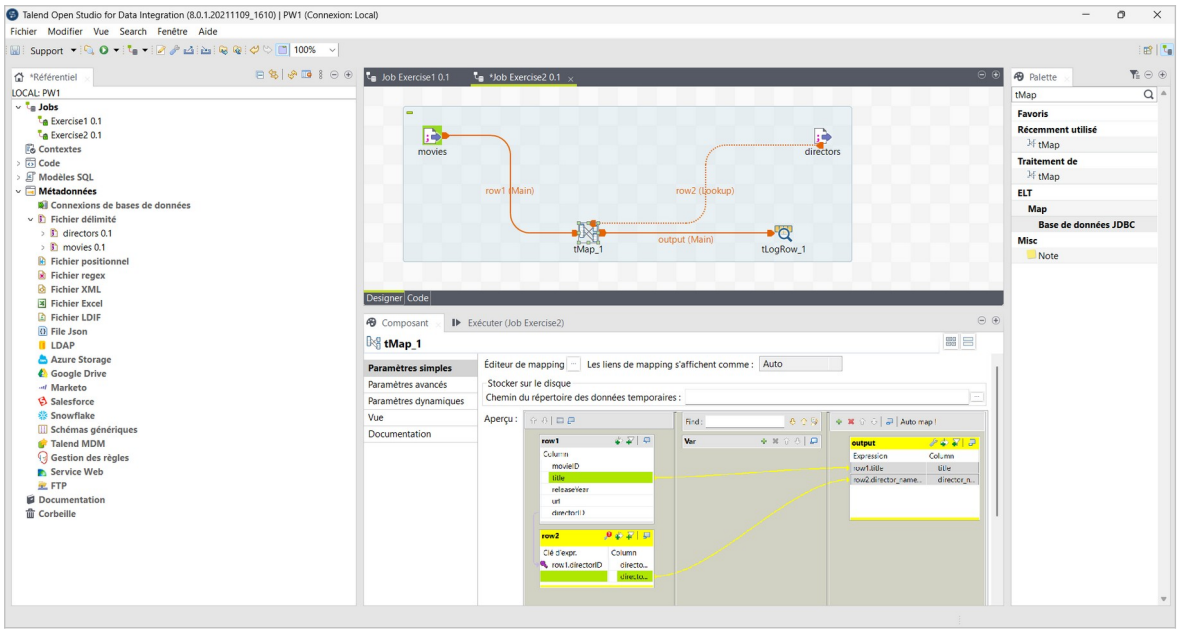



***Figure 8:*** *Job Exercise2 – movies (Main) and directors (Lookup) joined in the tMap, output sent to tLogRow*

## 3. Configuration of the tMap (steps 14 to 18)

- **Join (step 15):** `directorID` from row1 was dragged onto the **Expr. key** of `director_id` in row2 → join key `row1.directorID = row2.director_id`.
- **Join model (step 16):** in the row2 settings, **Join Model** was changed from Left Outer Join to **Inner Join**, so only movies that find a matching director are kept.
- **Output (step 17):** `row1.title` and `row2.director_name` were transferred to the output table. The expression `row2.director_name.trim()` was used to remove the leading/trailing spaces and tabs found in the source file (data cleaning of noisy values).

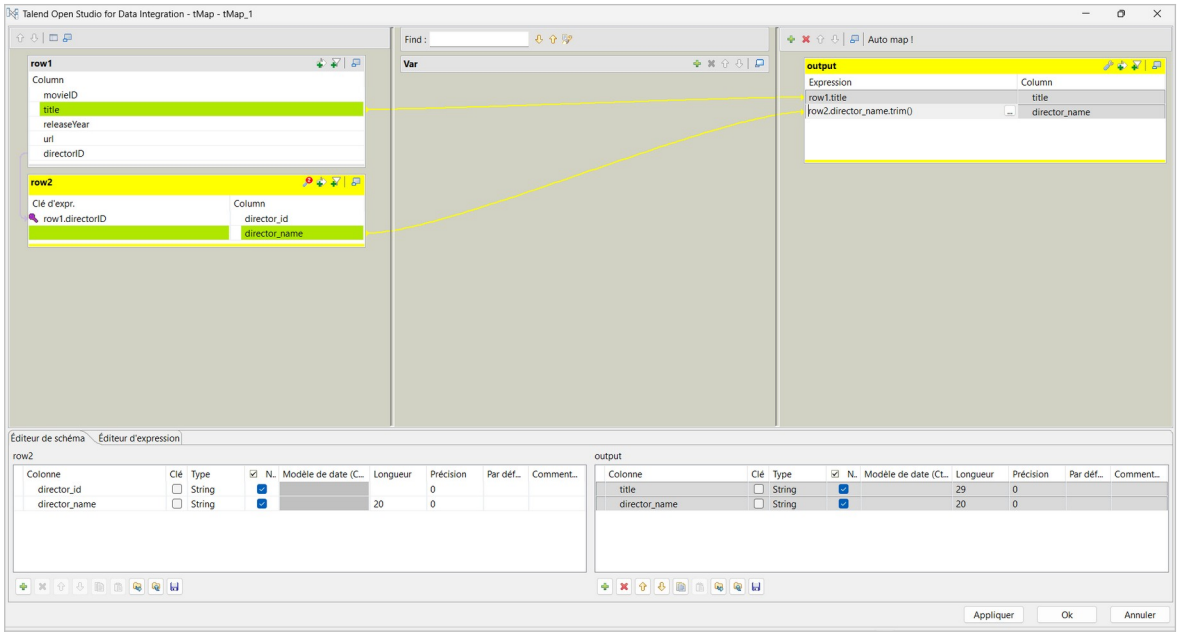



***Figure 9:*** *tMap editor – inner join on directorID = director_id and output columns title / director_name*

© 2026 Caplogy. All rights reserved.

## 4. Execution (step 19)

At the first execution, Talend asked to install a missing third-party library (**jboss-marshalling-river**, Apache-2.0 licence) used internally by the tMap lookup (tAdvancedHash). After accepting the licence and downloading it, the job ran successfully:

| Flow | Rows | Meaning |
|---|---|---|
| row1 (movies) | 1455 | All movies read from movies.csv |
| row2 (directors) | 204 | All directors loaded into the lookup |
| output | **255** | Movies with a valid director (inner join) |

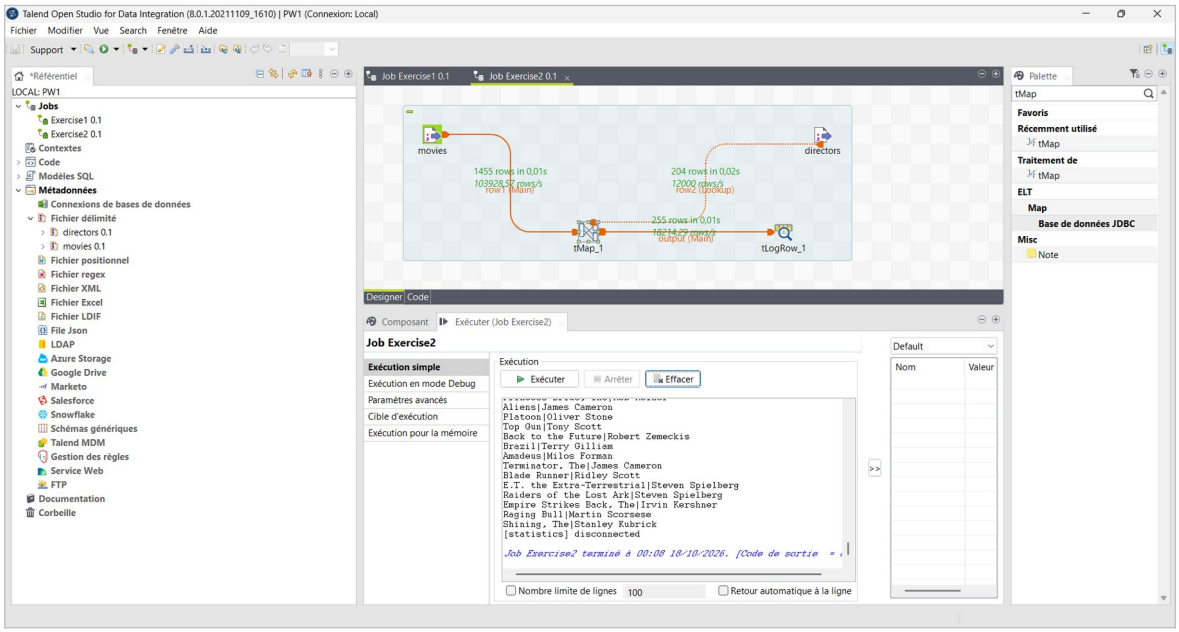



***Figure 10:*** *Execution of Exercise2: 1455 movies, 204 directors, 255 valid movies displayed (exit code 0)*

## 5. Job documentation (step 20)

| Component | Role | Main settings |
|---|---|---|
| movies (tFileInputDelimited) | Main input | Metadata `movies`, separator ";" |
| directors (tFileInputDelimited) | Lookup input | Metadata `directors`, separator ",", UTF-8, no header |
| tMap_1 | Transformation – join | Key `directorID = director_id`, Inner Join, unique match, load once |
| tLogRow_1 | Displays the result | Columns title, director_name |

> **Interpretation:** only **255 of the 1,455 movies (17.5 %)** have a director. The other **1,200 movies** have an empty `directorID` (missing data) and are removed by the Inner Join. This

© 2026 Caplogy. All rights reserved.

> job illustrates the **Transform** step of ETL: integration of two heterogeneous sources and filtering of incomplete records.

**Ehxercise 3:** This exercise describes how to filter the data stream (Movies) and obtain valid and invalid movies (use output files Load ETL step).

1. Create a Job Exercise 3:

2. Drag the movies file and choose TFileInputDelimited.

3. Drag the directors file and choose TFileInputDelimited.

4. Drag a TMap component.

5. Connect movies to TMap with a Main link.

6. Connect directors to TMap with a Main link.

7. Drag a TFileOutputDelimited component.

8. Drag a TFileOutputXML component.

9. Connect TMap to TFileOutputDelimited with a Main link named output_CSV.

10. Connect TMap to TFileOutputXML with a Main link named output_XML.

11. Double-click on the TMap.

12. Join the director_id column of movies with the director_id column of directors.

13. Set the JoinModel option to "Inner Join".

14. Transfer the title columns of movies and director name columns of directors to the output stream output_CSV.

15. Set the InnerJoinReject option to "false".

16. Transfer the title columns of movies to the output stream output_XML.

17. Set the InnerJoinReject option to "true".

18. Click Apply and OK.

19. Double-click on the TFileOutputDelimited and change the file name.

20. Double-click on the TFileOutputXML and change the file name. Execute the job Document the job

© 2026 Caplogy. All rights reserved.

## Answer – Exercise 3

### 1. Design of the job (steps 1 to 10)

The job **Exercise3** reuses the two input metadata (movies and directors) and one **tMap**. Two output components were added: a **tFileOutputDelimited** and a **tFileOutputXML**. The tMap is connected to them with two new Main outputs named **output_CSV** (order 1) and **output_XML** (order 2).

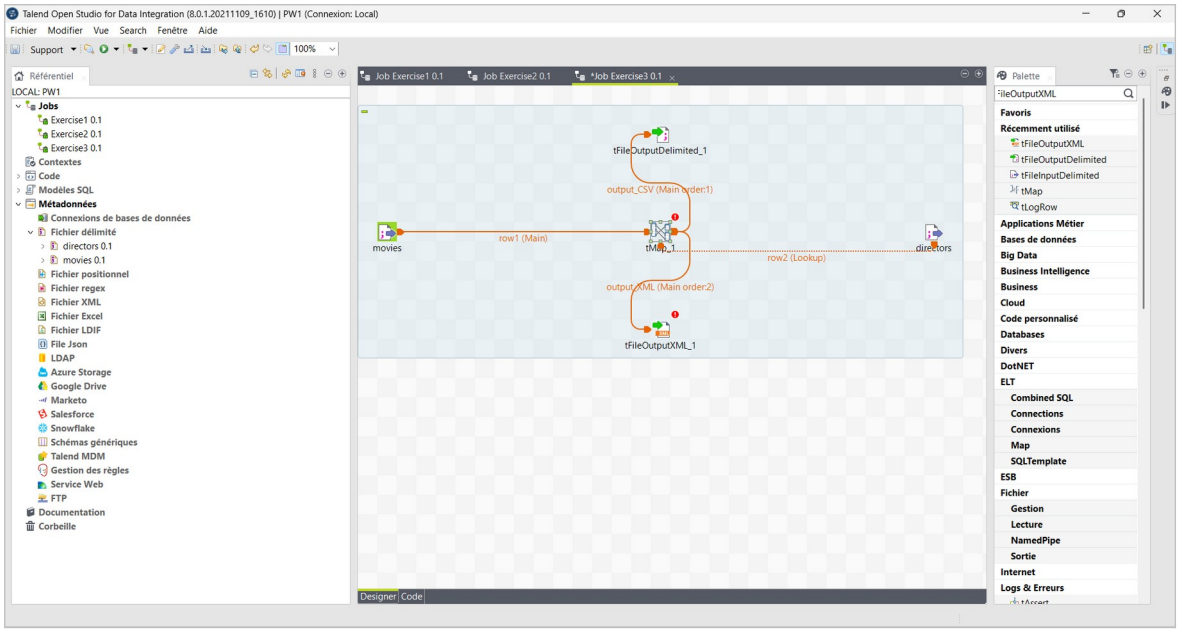



*Figure 11: Job Exercise3 – one tMap with two outputs: output_CSV and output_XML*

### 2. Configuration of the tMap (steps 11 to 18)

- **Join:** `row1.directorID = row2.director_id` with **Join Model = Inner Join** (steps 12–13).
- **output_CSV:** columns `row1.title` and `row2.director_name.trim()`; option **Catch lookup inner join reject = false** → receives the movies that **match** a director (steps 14–15).
- **output_XML:** column `row1.title`; option **Catch lookup inner join reject = true** → receives the movies **rejected** by the inner join, i.e. without a valid director (steps 16–17).

© 2026 Caplogy. All rights reserved.

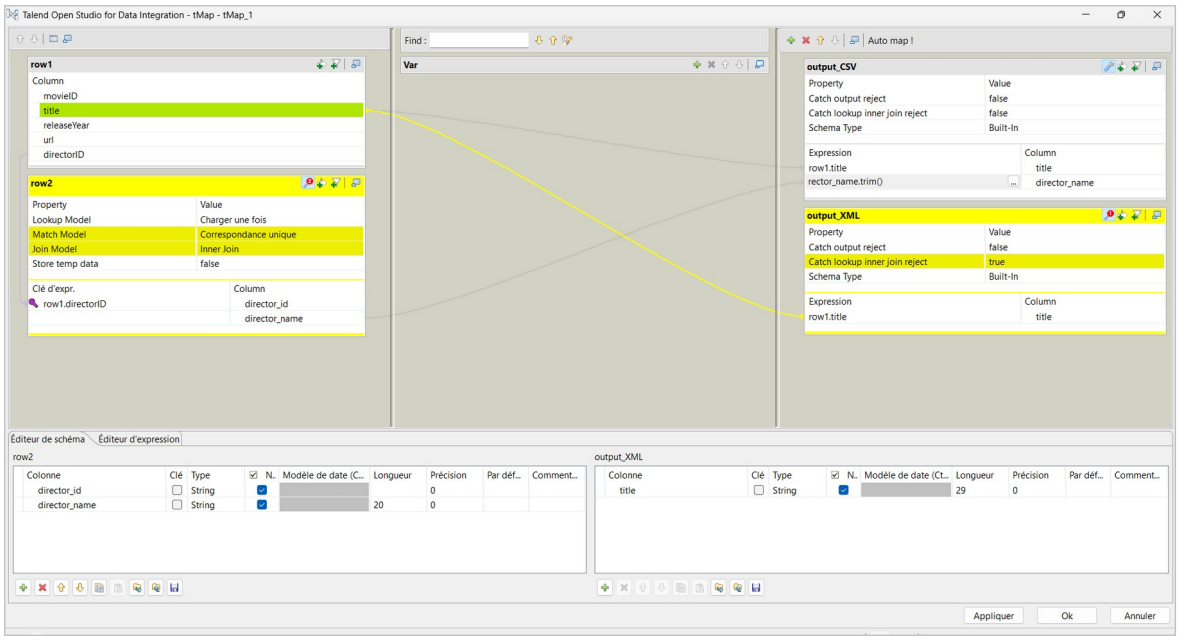



*Figure 12: tMap editor – output_CSV (reject = false) for valid movies and output_XML (reject = true) for rejected movies*

### 3. Output files (steps 19 and 20)

The file names were typed directly in the component settings (the Browse button only opens existing files):

- **tFileOutputDelimited:** …/`TalendLab5/valid_movies.csv`, field separator ";", option **Include header** enabled.
- **tFileOutputXML:** …/`TalendLab5/invalid_movies.xml`, row tag `"row"`.

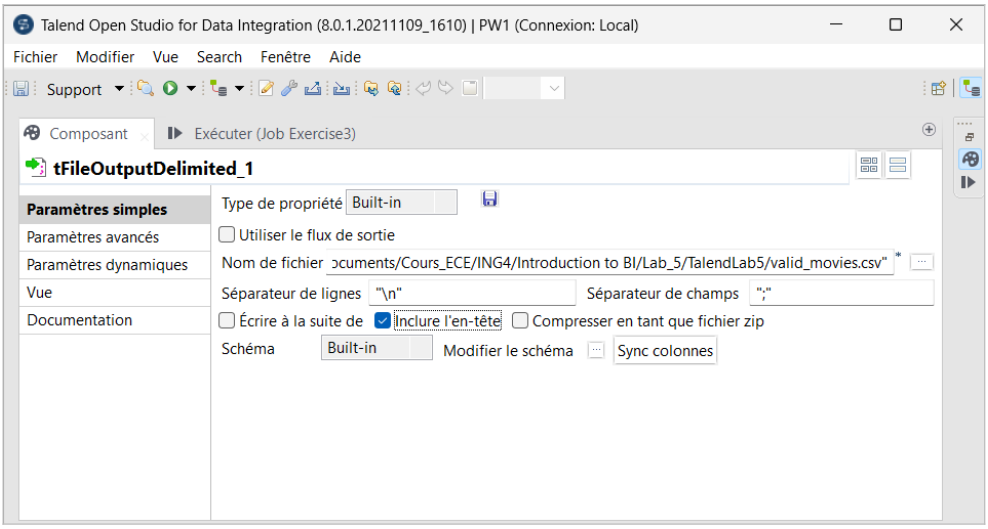



*Figure 13: tFileOutputDelimited settings – valid_movies.csv with header*

© 2026 Caplogy. All rights reserved.

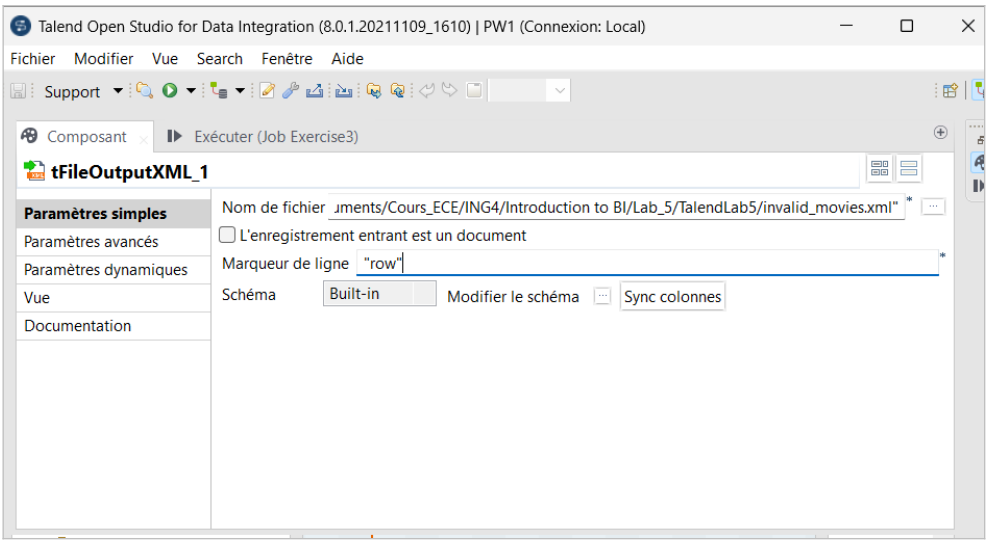



***Figure 14:*** *tFileOutputXML settings – invalid_movies.xml*

## 4. Execution

A first run stopped with the error “The particular file … valid_movies.csv already exist” (exit code 1), because the file had been created by an earlier attempt and the advanced option **Throw an error if the file already exist** was enabled. After unchecking this option, the file is simply overwritten and the job ends with **exit code 0**.

| Flow | Rows | Destination |
|---|---|---|
| row1 (movies) | 1455 | Main input |
| row2 (directors) | 204 | Lookup input |
| output_CSV | **255** | valid_movies.csv – valid movies with their director |
| output_XML | **1200** | invalid_movies.xml – rejected movies (no director) |

© 2026 Caplogy. All rights reserved.

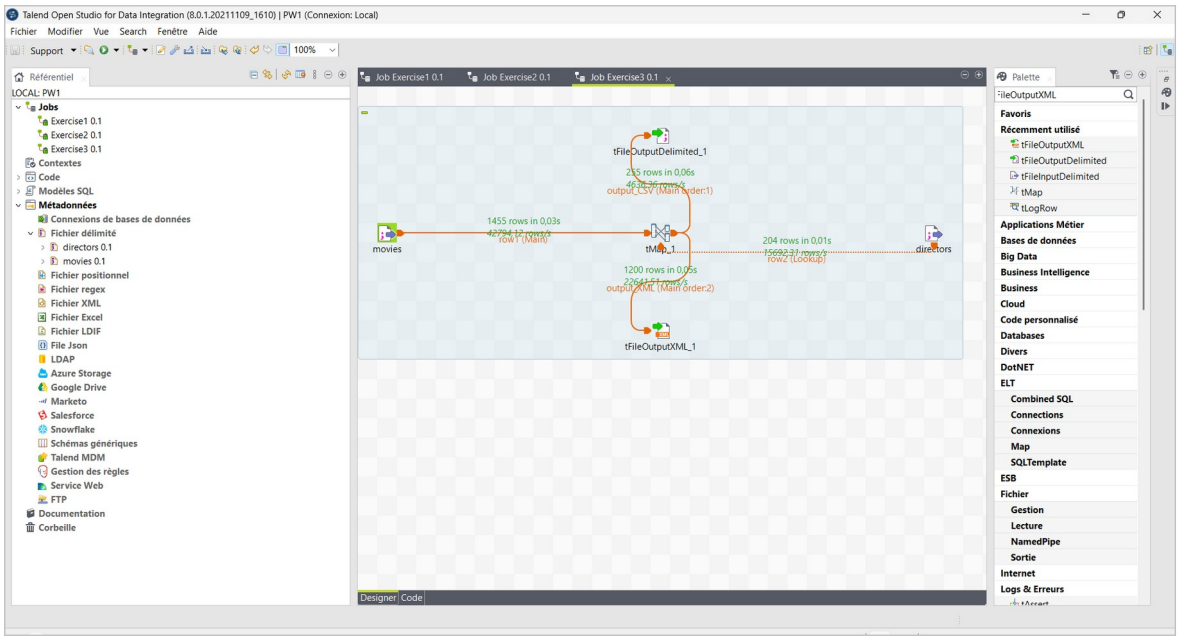



***Figure 15:*** *Execution of Exercise3: 255 valid movies to the CSV file and 1200 rejected movies to the XML file*

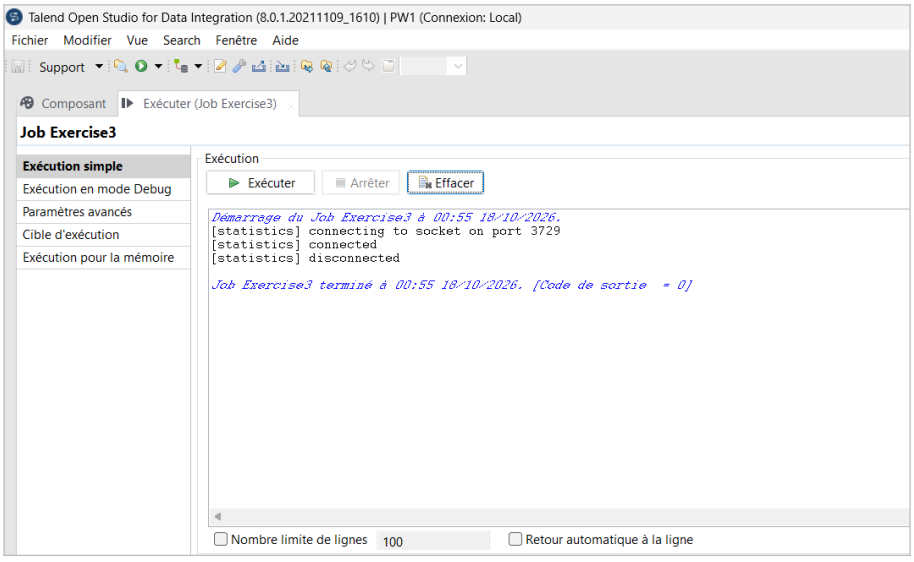



***Figure 16:*** *Run console of Exercise3 – job completed with exit code 0*

The check **255 + 1,200 = 1,455** confirms that every movie of the source ends up in exactly one of the two files: no record is lost or duplicated by the job.

## 5. Content of the generated files

**valid_movies.csv** (header + 255 lines, extract):

```
title;director_name
Apt Pupil;Bryan Singer
Titanic;James Cameron
Pulp Fiction;Quentin Tarantino
Shining, The;Stanley Kubrick
```

**invalid_movies.xml** (1,200 <row> elements, extract):

© 2026 Caplogy. All rights reserved.

```
<?xml version="1.0" encoding="ISO-8859-15"?>
<root>
  <row><title>Duoluo tianshi</title></row>
  <row><title>Great Expectations</title></row>
  <row><title>Cats Don&apos;t Dance</title></row>
  ...
</root>
```

## 6. Job documentation

| Component | Role | Main settings |
|---|---|---|
| movies / directors (tFileInputDelimited) | Main and lookup inputs | Repository metadata movies and directors |
| tMap_1 | Transformation and routing | Inner Join on directorID; output_CSV (reject = false); output_XML (reject = true) |
| tFileOutputDelimited_1 | Load – valid movies | valid_movies.csv, separator ";", header included |
| tFileOutputXML_1 | Load – rejected movies | invalid_movies.xml, row tag "row" |

> **Result:** this job illustrates the **Load** step of ETL. Instead of silently losing the incomplete records, the inner-join rejects are captured and stored in a separate file, so that they can be audited and corrected later (data governance).

© 2026 Caplogy. All rights reserved.

## Data Quality Observations and Conclusion

While building the three jobs, several data quality problems described in Chapter 4 were observed in the source files:

| Problem (Chapter 4) | Observation in the lab data | Treatment |
|---|---|---|
| Missing data | 1,200 of the 1,455 movies have an empty directorID | Rejected by the Inner Join and stored in invalid_movies.xml |
| Noisy data | Extra spaces and tabs around director names (e.g. " Gregg Araki", "  Alex Sichel") | Cleaned with `.trim()` in the tMap |
| Duplicated data | Repeated titles (e.g. Butcher Boy, Chairman of the Board, Hugo Pool) and Barbet Schroeder listed twice (IDs 13 and 14) | Identified; could be removed with a tUniqRow component |
| Conversion / encoding | Accented characters already lost in the sources (Cuar&#65533;n, Hallstr&#65533;m, C?r?monie, Mis?rables) | Not recoverable in Talend; must be fixed at the source |
| Format conflicts | Different separators (";" vs ","), header vs no header, UNIX vs WINDOWS line endings | Handled by two separate metadata definitions |

This lab covered the three steps of an ETL process with Talend Open Studio: **Extract** (reading a delimited file through a reusable metadata), **Transform** (joining two heterogeneous sources in a tMap with an Inner Join and cleaning the values) and **Load** (writing the valid movies to a CSV file and the rejected movies to an XML file). The final results (255 valid movies and 1,200 rejected movies, for a total of 1,455) are consistent with the source data, which confirms that the jobs are correct and that no record was lost.

© 2026 Caplogy. All rights reserved.